

Final Project Report

Ailoni Sahasransh Reddy

October 2025

Project Scope

This project explores how integer-only C code can be converted to use floating-point (FP) operations for improved parallelism and speedup, taking advantage of the fact that the FPU is idle or under-utilized. The aim is to exploit available FPUs to demonstrate measurable performance gains.

Objectives

- Analyze internal core architecture: Evaluate the number of integer and floating-point execution units, their respective latencies and throughput.
- Select three popular C programs operating on integer data and identify opportunities to convert computations to their FP equivalents wherever possible.
- Derive general guidelines for conversion and highlight potential limitations.

Implementation Steps

1. Literature Review and Architecture Analysis

- Study the microarchitecture for INTEL CPUs, identifying integer and FP pipeline resources, execution latencies, and throughput characteristics.
- Select three integer-data focused C programs.

2. Baseline Measurement

- Compile and execute the original integer-based code using optimization level -O3 on target systems.
- Collect results for execution time and power/energy consumption, using available hardware and software tools.

3. Conversion and Code Adaptation

- Analyze integer operations and identify segments suitable for FP conversion (e.g., loop computations and arithmetic expressions).
- Rewrite these segments using floating-point types and operations, carefully considering precision requirements.

- Test for correctness and numerical stability.

4. Guideline and Limitation Proposal

- Based on empirical results, derive actionable guidelines for when and how FP conversion is beneficial.
- Identify application scenarios and processor constraints where this approach may fail or result in undesirable side effects (e.g., energy penalty and numerical instability).

Verification Plan

- **Correctness:** Ensure output matches original integer-based code within acceptable numerical error bounds.
- **Performance:** Demonstrate speedup in execution time for atleast some target programs on INTEL hardware.
- **Optimization Consistency:** Use compiler optimization level -O3 as a standard.
- **Power/Energy:** Document changes in power/energy consumption by averaging over various runs using standard tools and compare improvement against overhead.
- **Documentation:** Log all changes, measurements, and architectural insights for reproducibility.

Proposed Guidelines

- Prefer FP conversion when:
 - FPU resources are idle or under-utilized.
 - Application is throughput-bound, not precision-critical.
 - FP units offer higher parallelism or lower latency.
- Avoid FP conversion when:
 - Integer operations are inherently faster or FP conversion increases overhead.
 - Numerical stability/precision is essential.
 - Energy cost outweighs performance gain.

Limitations

- FPUs may not always yield speedup for every workload; overhead from type conversion, pipeline stalls, and numerical instability must be considered.
- ARM and Intel architectures differ in pipeline structure and FPU/ALU sharing, affecting portability of results.
- Energy efficiency may degrade, especially for high-throughput FP operations if not managed carefully.

Current Status

- Finished setting up gem5 simulator to assist in analysing performance.
- Experimented in achieving speedup for:
 - Finding maximum element of an array.(21% speedup)
 - Performing matrix multiplication.(slight speedup)
 - Performing dot product of two vectors.(slight speedup)

Challenges

- Achieving speedup in -O3 optimization.
- Identifying apt approach in order to obtain maximum speedup.

Current results

- Achieved 21% speedup for:
 - Finding maximum element of an array.
- Achieved minimal speedup(2-5 %) for:
 - Performing matrix multiplication.
 - Performing dot product of two vectors.

Finding maximum element of an array

Key Features

- Applied loop unrolling to obtain more benefit of using floating point operations.
- Multiple accumulators have not been used in this code, the effects of multiple accumulators have been analysed in further codes.
- The gem5 simulator has been used to measure the time clocked in this code.
- This is a minimalistic way to achieve a good speedup.

Experimenting on various unrolls and ratios

All the results depicted in this section correspond to the same unroll factor in both integer and floating point codes. The data thus shows the speedup attained exclusively for floating point operations.

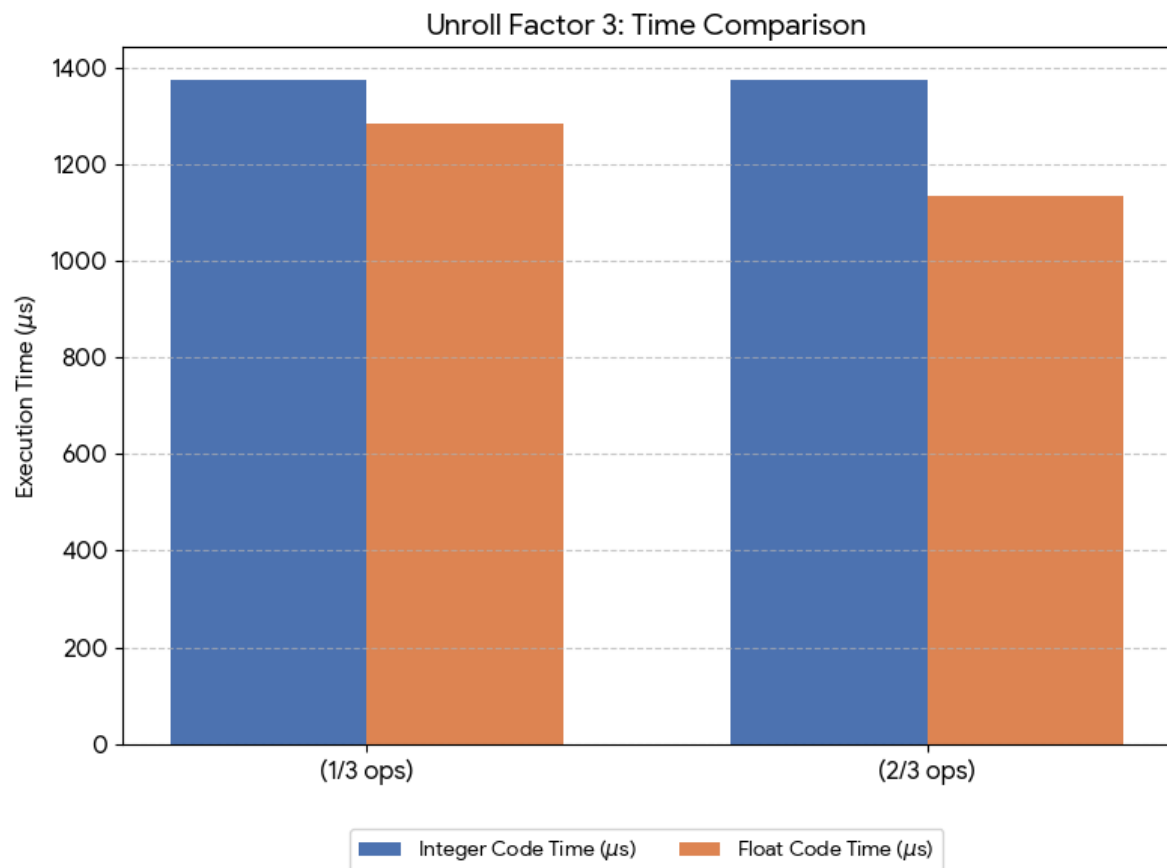
All the values are in microseconds(denoted by q). The experiments are done on a randomized array of size 10000 as it takes roughly 10 seconds to run on the simulator currently. Also, the values depicted are only for the optimized part of finding the maximum element of the array, the remaining values(such as initialization) have been duly subtracted before reporting.

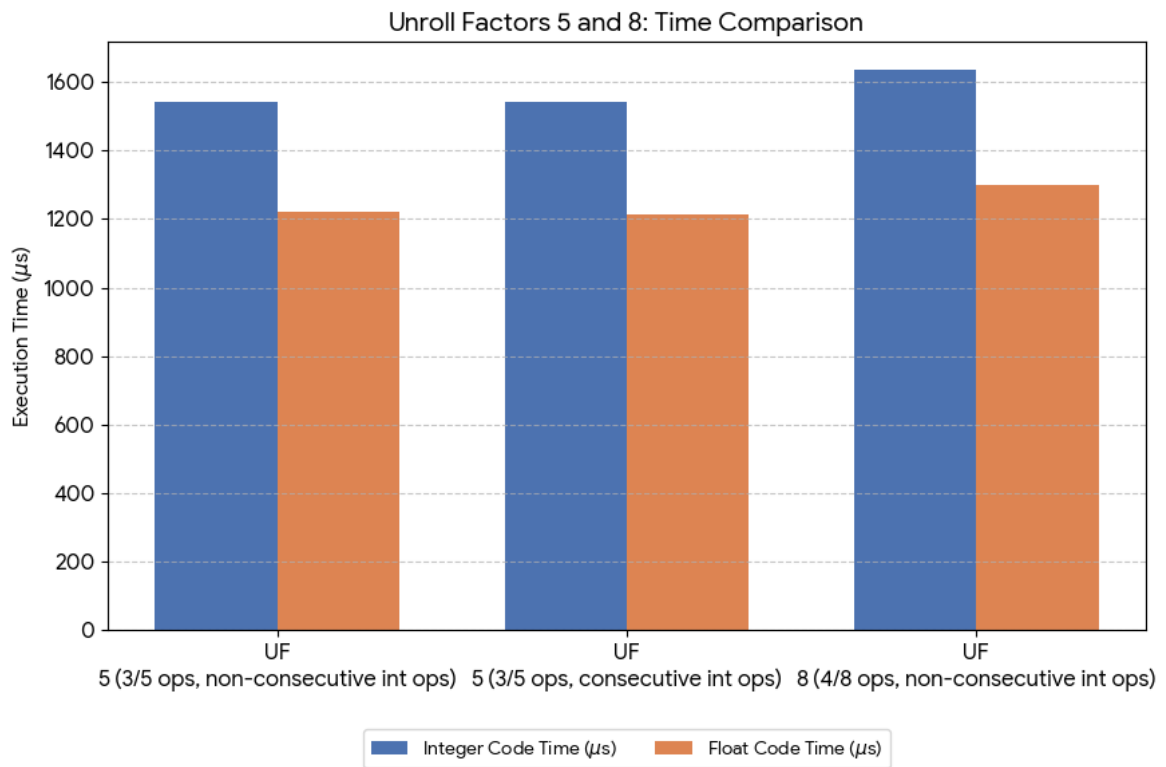
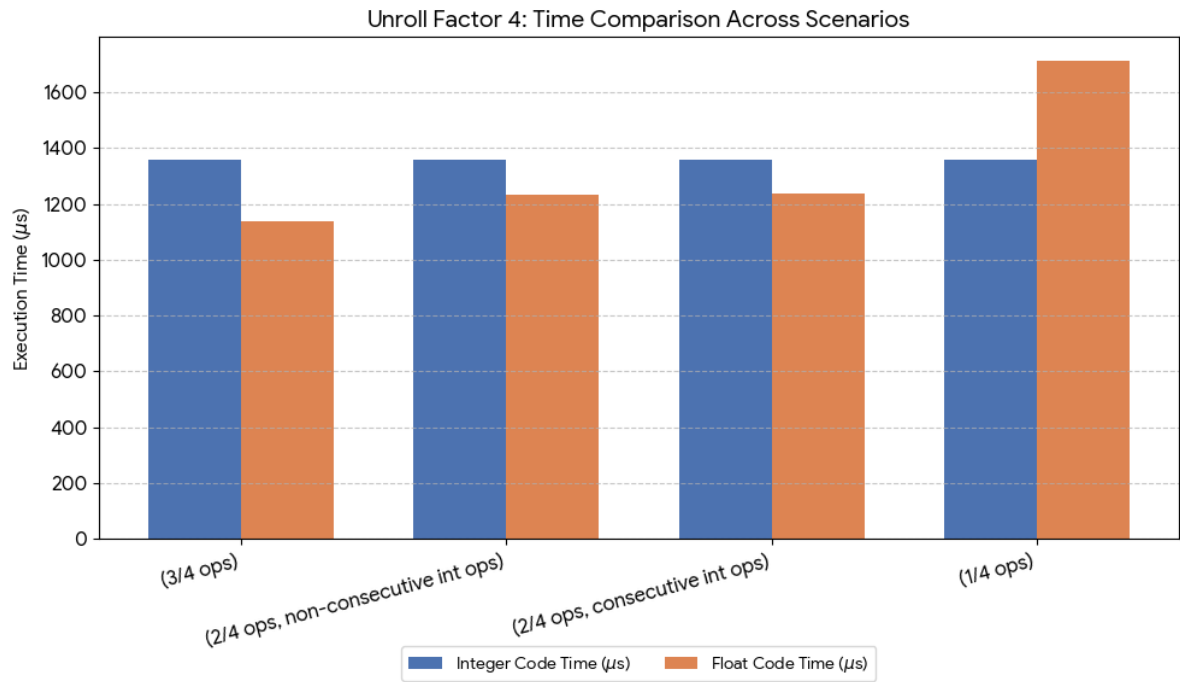
- For 3 unrolls, if float operations are used in 1/3 of the code, the integer code takes 1375q while the float code takes 1284q, resulting in a 7% speedup.
- For 3 unrolls, if float operations are used in 2/3 of the code, the integer code takes 1375q while the float code takes 1134q, resulting in a 21% speedup.
- For 4 unrolls, if float operations are used in 3/4 of the code, the integer code takes 1360q while the float code takes 1138q, resulting in a 20% speedup.
- For 4 unrolls, if float operations are used in 2/4 of the code such that the 2 integer operations are always non-consecutive, the integer code takes 1360q while the float code takes 1232q, resulting in a 10% speedup.
- For 4 unrolls, if float operations are used in 2/4 of the code such that the 2 integer operations are always consecutive, the integer code takes 1360q while the float code takes 1239q, resulting in a 10% speedup.
- For 4 unrolls, if float operations are used in 1/4 of the code, the integer code takes 1360q while the float code takes 1712q, resulting in a slowdown.
- For 5 unrolls, if float operations are used in 3/5 of the code such that the 2 integer operations are always non-consecutive, the integer code takes 1542q while the float code takes 1222q, resulting in a 26% speedup.
- For 5 unrolls, if float operations are used in 3/5 of the code such that the 2 integer operations are always consecutive, the integer code takes 1542q while the float code takes 1212q, resulting in a 27% speedup.
- For 8 unrolls, if float operations are used in 4/8 of the code such that any 2 of the 4 integer operations are always non-consecutive, the integer code takes 1634q while the float code takes 1298q, resulting in a 26% speedup.

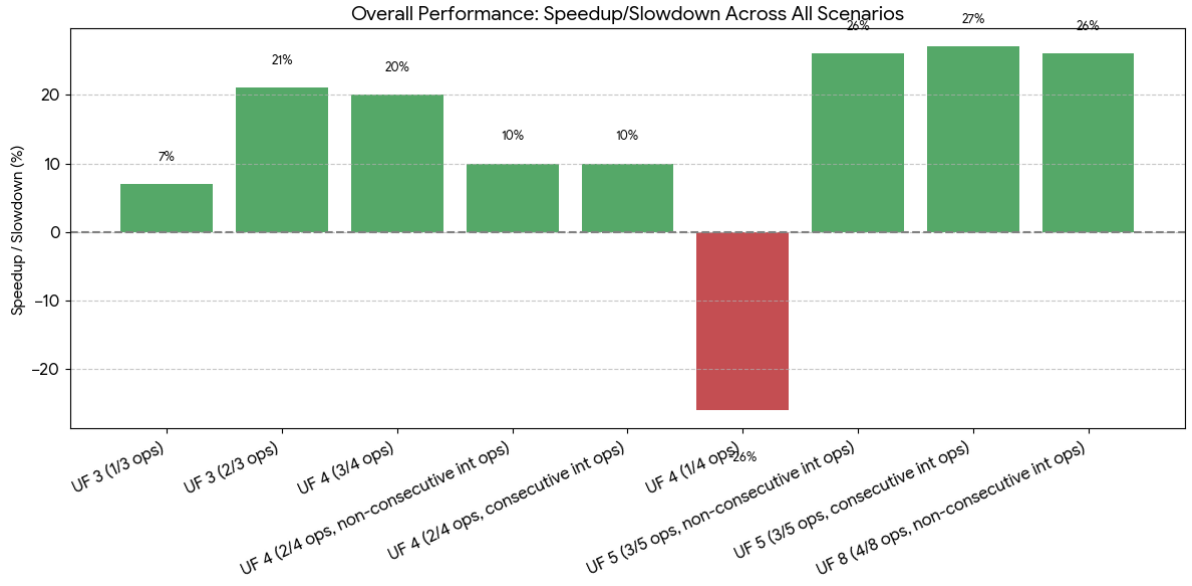
The above are some of the results obtained. The 5 unrolls and 8 unrolls cases are giving higher speedup due to the integer slowing down, hence the optimal speedup obtained so far is 21%.

Loop Unrolling Performance Metrics (μs)

Unroll Factor (UF)	Float Ops Used	Integer Code (μs)	Float Code (μs)	Speedup / Slowdown (%)	Descriptor
3	1/3	1375	1284	7% (Speedup)	1/3 ops
3	2/3	1375	1134	21% (Speedup)	2/3 ops
4	3/4	1360	1138	20% (Speedup)	3/4 ops
4	2/4	1360	1232	10% (Speedup)	2/4 non-consecutive int ops
4	2/4	1360	1239	10% (Speedup)	2/4 consecutive int ops
4	1/4	1360	1712	-26% (Slowdown)	1/4 ops
5	3/5	1542	1222	26% (Speedup)	3/5 non-consecutive int ops
5	3/5	1542	1212	27% (Speedup)	3/5 consecutive int ops
8	4/8	1634	1298	26% (Speedup)	4/8 non-consecutive int ops







Observations

- The best time clocked for integer code has 4 unrolls, hence, this is the benchmark which we need to use for computing speedup.
- The best speedup obtained with respect to the benchmark set is 20% for 3/4 floating operations.
- The unrolling factor need to be a power of 2, otherwise some lanes are wasted during computation, thus leading to suboptimal results(especially for integer code).
- Moreover, the registers run out as the unroll factor increases, thus leading to increase in running time for integer code(integer only code uses only 32 registers, so it runs out quicker, whereas float code uses 32 other floating point registers as well).
- The reason for 1/4 ratio giving a slowdown could be that the compiler is unable to vectorize in SIMD.

```

codes > C 1f.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define n 100000
5  static int data[n];
6
7  int main() {
8      srand(42);
9
10     for(int i=0;i<n;i++) {
11         data[i]=rand()%10000;
12     }
13     float max2=0.0f;
14     int max1=0;
15     for(int i=0;i<n;i+=4) {
16         max2=(max2>(float)data[i])?max2:(float)data[i];
17         max2=(max2>(float)data[i+1])?max2:(float)data[i+1];
18         max2=(max2>(float)data[i+2])?max2:(float)data[i+2];
19         max1=(max1>data[i+3])?max1:data[i+3];
20     }
21     printf("%d\n", ((int)max2>max1)?(int)max2:max1);
22 }

```

Variations in size of array

- A good speedup still exists, meaning that the obtained data isn't an outlier.

Performance vs. Array Size (Sorted)

Array Size	Integer Code Time (μ s)	Float Code Time (μ s)	Speedup / Slowdown (%)
10^4	137	115	19% (Speedup)
10^5	1360	1138	20% (Speedup)
10^6	15772	13449	17% (Speedup)

Performing matrix multiplication

Key Features

- This code has been inspired from the source OpenBLAS.
- Applied loop unrolling to obtain more benefit of using floating point operations.
- Multiple accumulators have been used in this code.
- The concept of tiling has been implemented in this code.
- This code is cache friendly.
- Intel hardware has been used to measure the time clocked in this code.
- This is a more advanced way to achieve a good speedup compared to the previous code.
- As the values in the matrix are going to change everytime, unlike in previous code, each value depicted is an average of 10 runs.
- On reading documentation by high performance libraries, I have set an unroll factor of 16 and 4 accumulators for uniformity across all experiments.
- All the experiments have been performed on square matrices of size n .

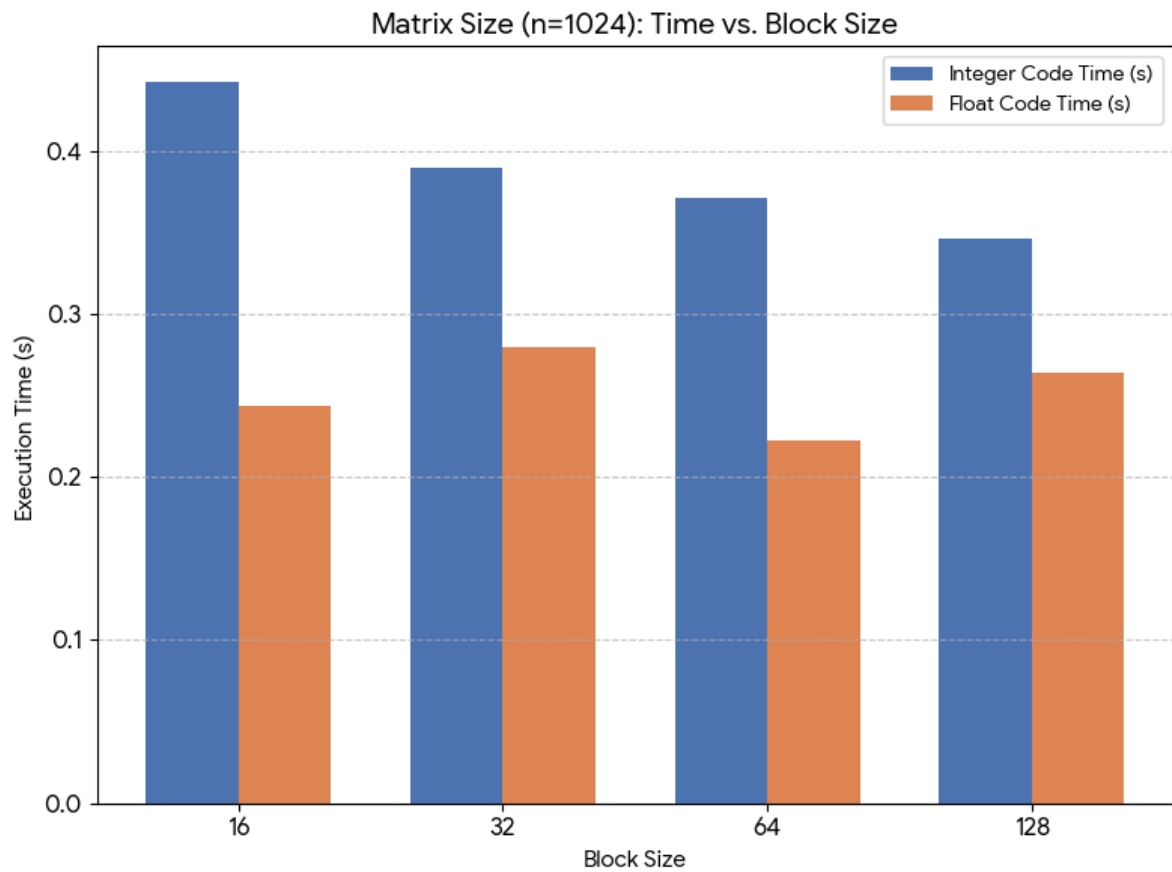
Experimenting on various matrix sizes and block sizes

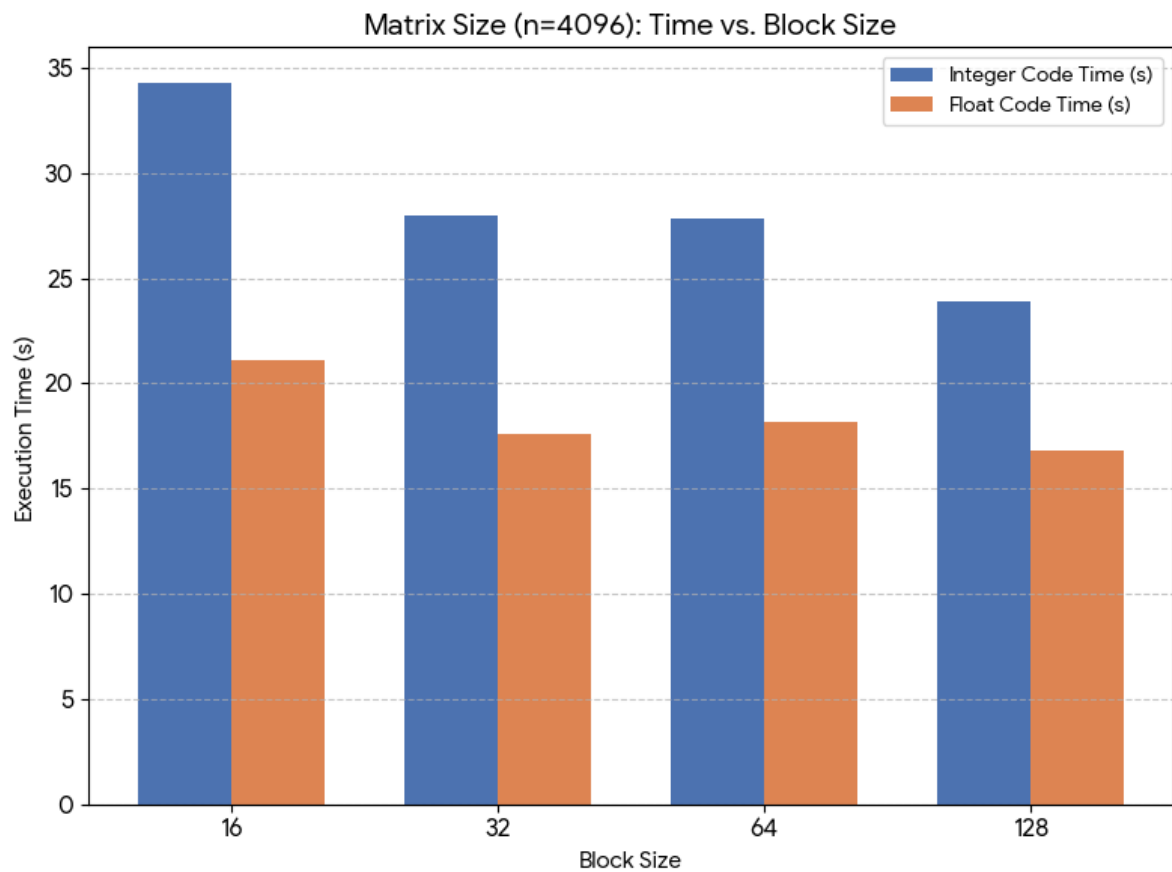
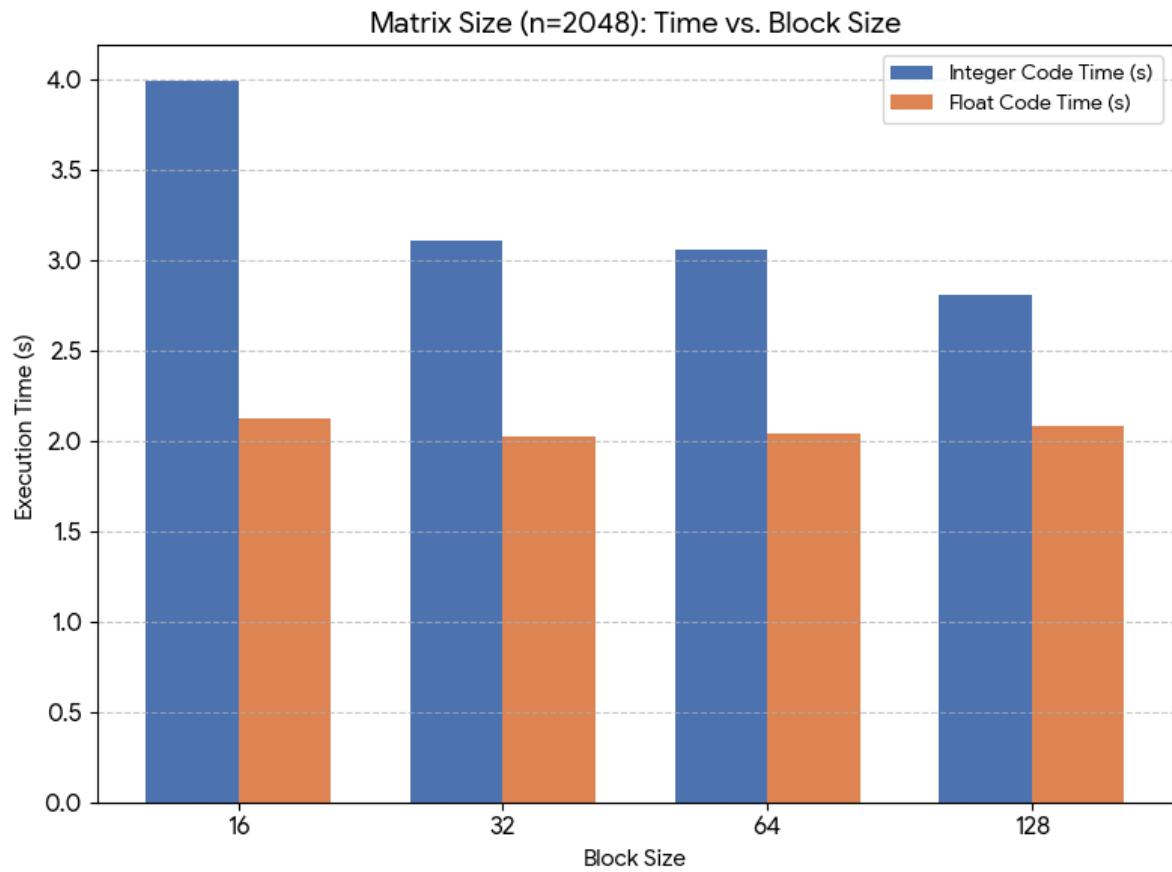
All the results depicted in this section correspond to the same unroll factor in both integer and floating point codes. The data thus shows the speedup attained exclusively for floating point operations.

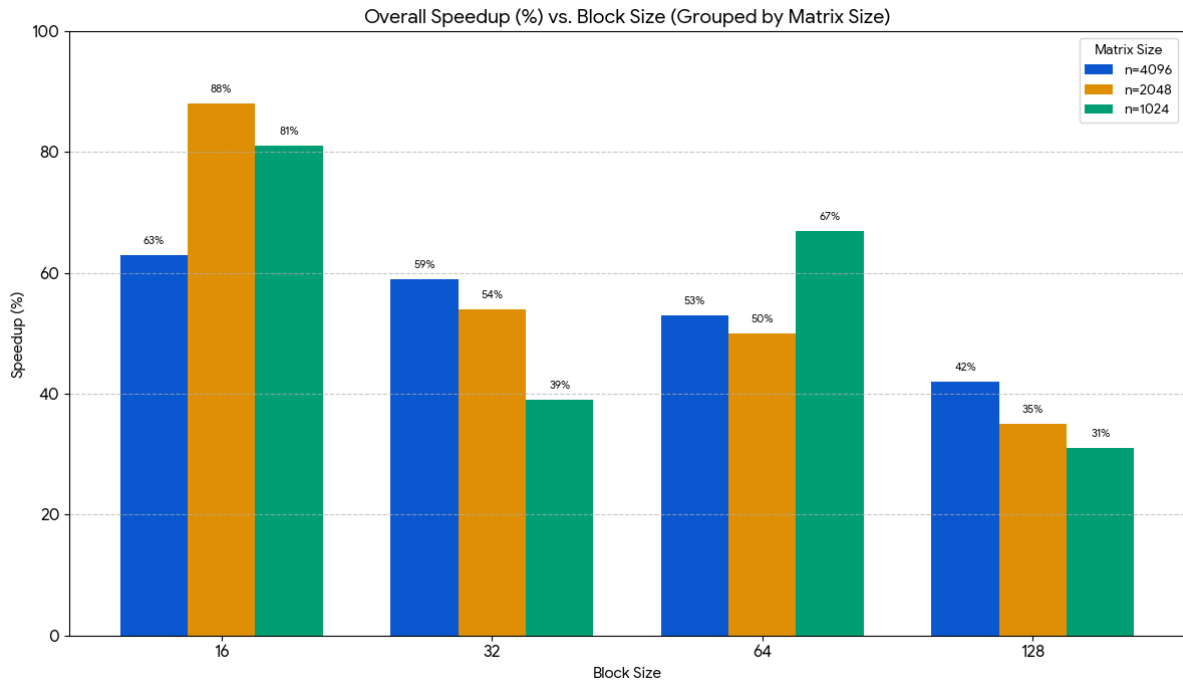
All the values are in seconds(denoted by s). Also, the values depicted are only for the the optimized part of performing matrix multiplication, the remaining values(such as initialization) have been duly subtracted before reporting.

Matrix Multiplication Performance: Block Size vs. Speedup

Matrix Size (n)	Block Size	Integer Code Time (s)	Float Code Time (s)	Speedup / Slowdown (%)
4096	16	34.270	21.080	63% (Speedup)
4096	32	27.970	17.600	59% (Speedup)
4096	64	27.860	18.170	53% (Speedup)
4096	128	23.870	16.780	42% (Speedup)
2048	16	3.990	2.120	88% (Speedup)
2048	32	3.110	2.020	54% (Speedup)
2048	64	3.060	2.040	50% (Speedup)
2048	128	2.810	2.080	35% (Speedup)
1024	16	0.442	0.244	81% (Speedup)
1024	32	0.390	0.280	39% (Speedup)
1024	64	0.371	0.222	67% (Speedup)
1024	128	0.346	0.264	31% (Speedup)







Observations

- The larger the block size, the better the cache usage.
- When block size is 16 or 32, most of the cache accesses are from L1.
- When block size is 64 or 128, most of the cache accesses are from L2.
- Block sizes of 256 and higher don't fit in both L1 and L2.
- Block sizes of 8 and lower have very poor performance in terms of locality.
- For small sized matrices, the block size doesn't make a lot of difference.
- For medium sized matrices, 64-sized blocks give best performance.
- For large sized matrices, 128-sized blocks give best performance, as more elements are accommodated in the L2 cache.

Performing dot product of two vectors

Key Features

- This code has been inspired from the source OpenBLAS.
- Applied loop unrolling to obtain more benefit of using floating point operations.
- Multiple accumulators have been used in this code.
- This code is cache friendly.
- Intel hardware has been used to measure the time clocked in this code.

- This is a simpler way to achieve a good speedup compared to the previous code.
- As the values in the vectors are going to change everytime, like in previous code, each value depicted is an average of 10 runs.
- On reading documentation by high performance libraries, I have set an unroll factor of 16 and 4 accumulators for uniformity across all experiments.
- This code deals with dot product of two vectors, and not multiple.
- Dot product of multiple vectors can be converted into a matrix multiplication.

Experimenting on various vector sizes

All the results depicted in this section correspond to the same unroll factor in both integer and floating point codes. The data thus shows the speedup attained exclusively for floating point operations.

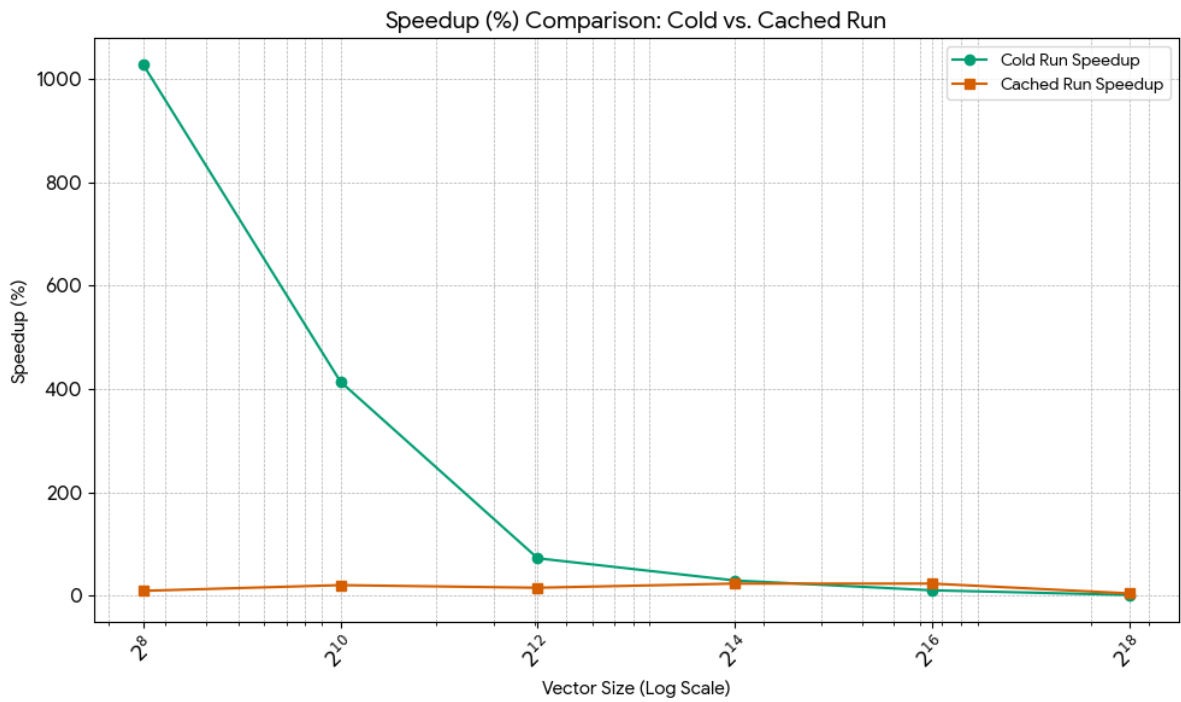
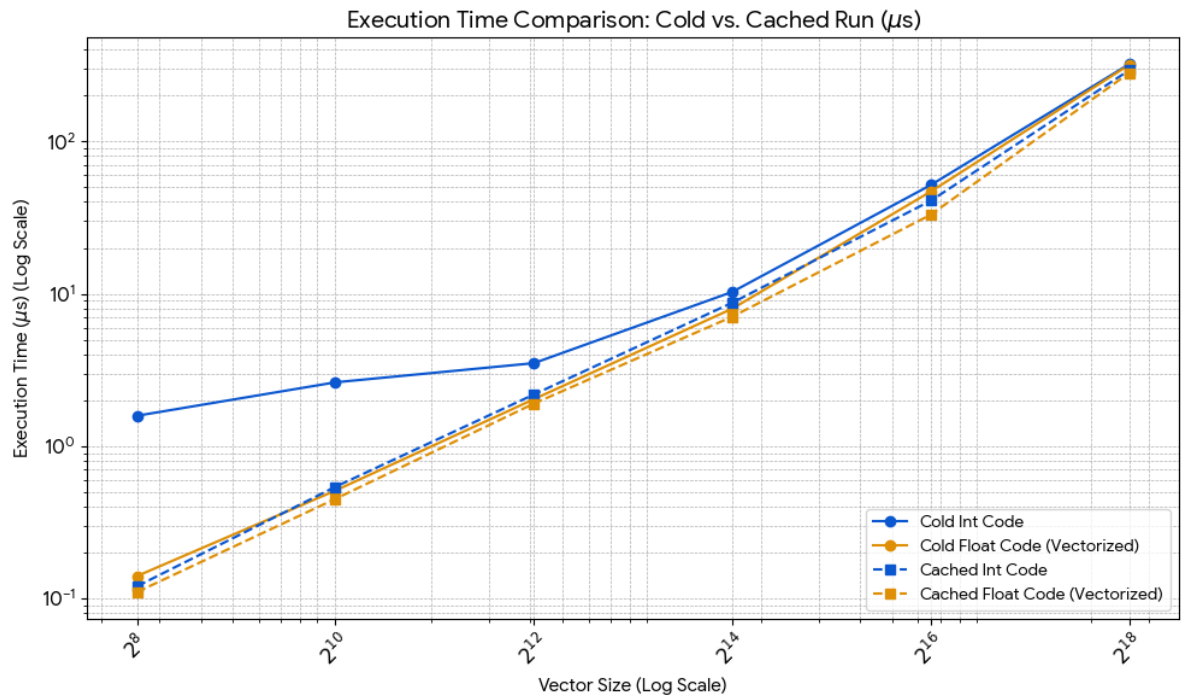
All the values are in microseconds. Also, the values depicted are only for the the optimized part of performing dot product of two vectors, the remaining values(such as initialization) have been duly subtracted before reporting.

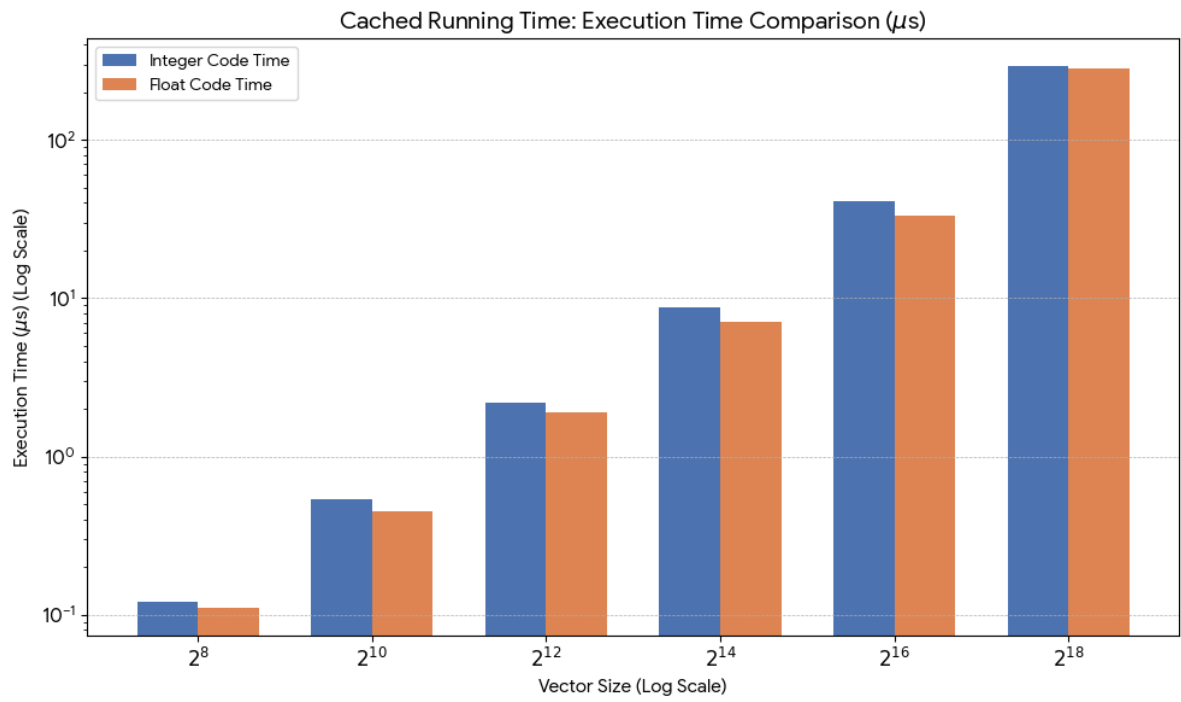
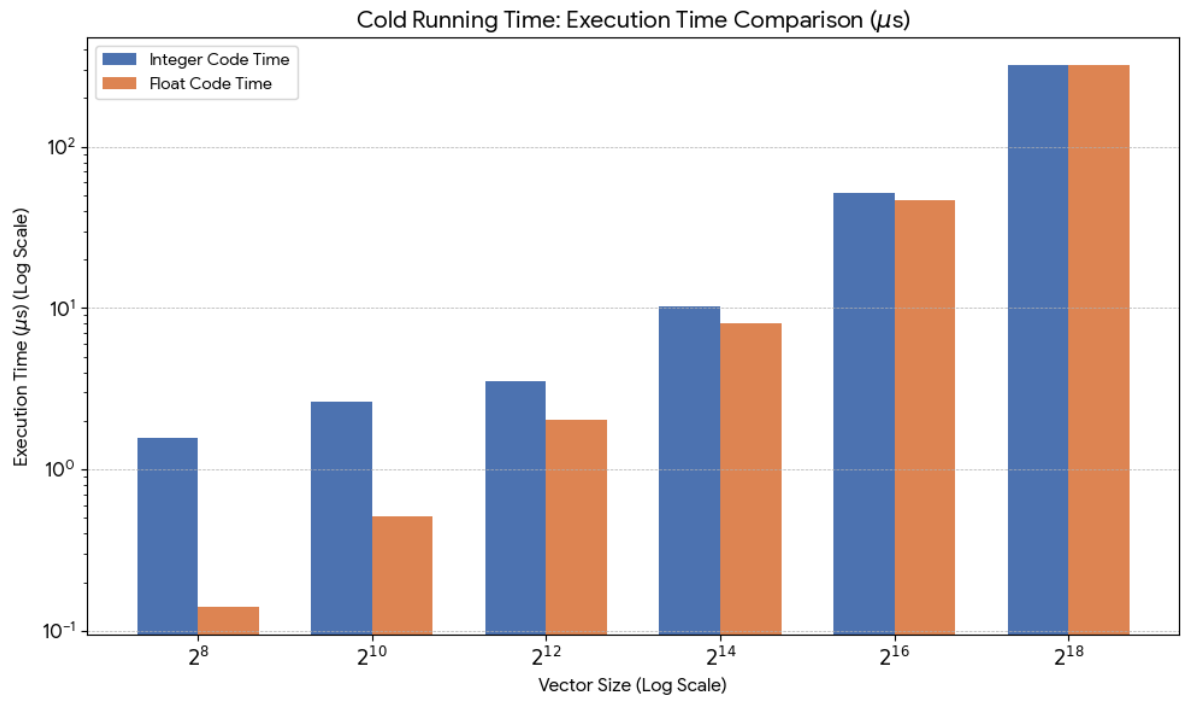
Cold Running Time

Vector Size	Integer Code Time (μs)	Float Code Time (μs)	Speedup / Slowdown (%)
256	1.58	0.14	1028% (Speedup)
1024	2.62	0.51	414% (Speedup)
4096	3.50	2.03	72% (Speedup)
16384	10.28	7.98	29% (Speedup)
65536	51.71	46.86	10% (Speedup)
262144	321.57	318.10	1% (Speedup)

Cached Running Time

Vector Size	Integer Code Time (μs)	Float Code Time (μs)	Speedup / Slowdown (%)
256	0.12	0.11	9% (Speedup)
1024	0.54	0.45	20% (Speedup)
4096	2.18	1.90	15% (Speedup)
16384	8.73	7.07	23% (Speedup)
65536	40.76	33.02	23% (Speedup)
262144	293.61	281.53	4% (Speedup)





Observations

- For smaller vector sizes(less than 2048), the entire computation is compute-bound, relying entirely on vectorized float being faster than scalar int.
- For medium vector sizes(2048-16384), the cache accesses get into L2, thus the code is both compute-bound and memory-bound.
- For large vector sizes(greater than 16384), the cache accesses override L2, the code is entirely memory-bound.
- Cached time is similar for both int and float when the vector size is small as the code is compute-bound entirely.
- Cached time approaches cold time as size of vector increases as memory accesses almost entirely take place from RAM at higher vector sizes.
- As vector sizes increase, we need to fall back on the techniques used in the previous code like tiling/blocking.

Final Summary

- The three tasks have been selected and performed in such a way that the principal concepts used in each are different.
- The tasks in increasing order of complexity are finding maximum element of an array, performing dot product of two vectors, and performing matrix multiplication.
- It may seem that dot product of two vectors is just a special case of matrix multiplication, but it is in practice not required to run a generalized case always as we aim to make the common case faster in architecture. As dot product of two vectors is encountered way too often in real scenarios, this code aims to simplify the order of optimization.